

Confluent/KsqlDB and data integration

Set up

Installez Docker desktop si vous ne l'avez pas encore: <https://www.docker.com/products/docker-desktop/>

Clonez ou téléchargez le répertoire Git de confluent

Pour cloner:

Git clone <https://github.com/confluentinc/cp-all-in-one.git>

Pour télécharger

<https://github.com/confluentinc/cp-all-in-one>

Exécutez la commande

```
docker compose up -d
```

Vérifiez que tous vos container aient bien démarré. Il est possible que certains ne démarrent pas correctement, relancez les.

Ouvrir le control center

●	cp-all-in-one	-	-	-	15.56%	2 days ago	■	:	🗑
●	ksql-datagen	9364d619323e	confluentinc		0.01%	2 days ago	■	:	🗑
●	ksqldb-cli	09491e05fed3	confluentinc		0%	2 days ago	■	:	🗑
●	control-center	d66da719465e	confluentinc	9021:9021 ↗	4.51%	2 days ago	■	:	🗑
●	ksqldb-server	61be5ec03ffa	confluentinc	8088:8088 ↗	3.94%	2 days ago	■	:	🗑
●	rest-proxy	8bedce57076e	confluentinc	8082:8082 ↗	0.48%	2 days ago	■	:	🗑
●	connect	fb514c38aa31	cnfldemos/	8083:8083 ↗	1.32%	2 days ago	■	:	🗑
●	schema-registry	d30a4a16bc89	confluentinc	8081:8081 ↗	0.49%	2 days ago	■	:	🗑
●	broker	d7c953fbb4e4	confluentinc	9092:9092 ↗ Show all ports (2)	4.59%	2 days ago	■	:	🗑
●	zookeeper	3c6c5ebb713d	confluentinc	2181:2181 ↗	0.22%	2 days ago	■	:	🗑

Click here

Python

Il est nécessaire que sur vos machines, vous ayez Python 3 installé. Si ça n'est pas le cas, faites-le.

Depuis votre terminal:

- Vérifiez que vous avez python installé en faisant: **python --version**
- Vérifiez que vous avez pip installé en faisant: **pip --version**
- Installez le package de gestion de Kafka en faisant: **pip install kafka-python-ng**

I - Prise en main

1- Input Kafka topic

Créer un topic Kafka ayant pour nom: **transaction_log**

2- Analyse et compréhension des messages de logs

Exemple de structure d'un message de transaction

```
{'transaction_id': 'TXN-87a09c4b', 'timestamp': '2025-02-23T16:39:23.364214Z', 'user_id': 'USER-7331', 'user_name': 'Campbell', 'product_id': 'PROD-928', 'amount': 83.28, 'currency': 'AUD', 'transaction_type': 'refund', 'status': 'completed', 'location': {'city': 'Chennai', 'country': 'India'}, 'payment_method': 'apple_pay', 'product_category': 'books', 'quantity': 4, 'shipping_address': {'street': '450 Main St', 'zip': '51715', 'city': 'Chennai', 'country': 'India'}, 'device_info': {'os': 'Linux', 'browser': 'Chrome', 'ip_address': '120.168.78.193'}, 'customer_rating': None, 'discount_code': 'DISCOUNT-174', 'tax_amount': 16.38, 'thread': 2, 'message_number': 99, 'timestamp_of_reception_log': '13/03/2025 17:34:20'}
```

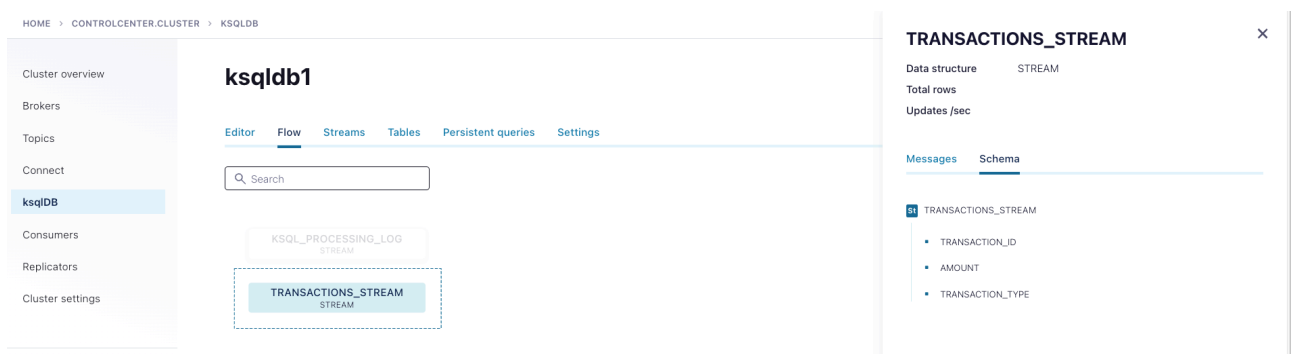
3- Création d'un stream

Exécutez la commande ci-dessous. Dans cet exemple de prise de main, nous ne nous intéressons qu'à un nombre limité de champs.

Question: Décrire ce que fait la commande ligne par ligne

```
CREATE STREAM transactions_stream (  
  transaction_id STRING,  
  amount DOUBLE,  
  transaction_type STRING  
) WITH (  
  KAFKA_TOPIC='transaction_log',  
  VALUE_FORMAT='JSON'  
);
```

Sous l'onglet Flow, vous devrez voir votre nouveau stream crée comme le montre l'image ci-dessous



4- Création d'un nouveau stream filtré

Créez un nouveau stream pour les transactions importantes (dont le montant est supérieur à 700)

```
CREATE STREAM transactions_importantes_700 AS  
SELECT * FROM transactions_stream WHERE amount > 700;
```

Sous l'onglet Flow, on voit bien le pipeline qui se met en place. En cliquant sur "CREATE-STREAM", en haut à droite, on peut voir la requête qui a été exécuté



```
CREATE STREAM TRANSACTIONS_IMPORTANTES_700 WITH (CLEANUP_POLICY='delete',  
KAFKA_TOPIC='TRANSACTIONS_IMPORTANTES_700', PARTITIONS=1, REPLICAS=1,  
RETENTION_MS=604800000) AS SELECT *  
FROM TRANSACTIONS_STREAM TRANSACTIONS_STREAM  
WHERE (TRANSACTIONS_STREAM.AMOUNT > 700)  
EMIT CHANGES;
```

En effet, un nouveau topic 'TRANSACTIONS_IMPORTANTES_700' a été créé avec les caractéristiques du topic d'entrée, l'opération de filtre est effectuée et le résultat est écrit dans le topic nouvellement créé. `CLEANUP_POLICY='delete'` implique que les vieux messages qui auront dépassé leur période de rétention seront supprimés.

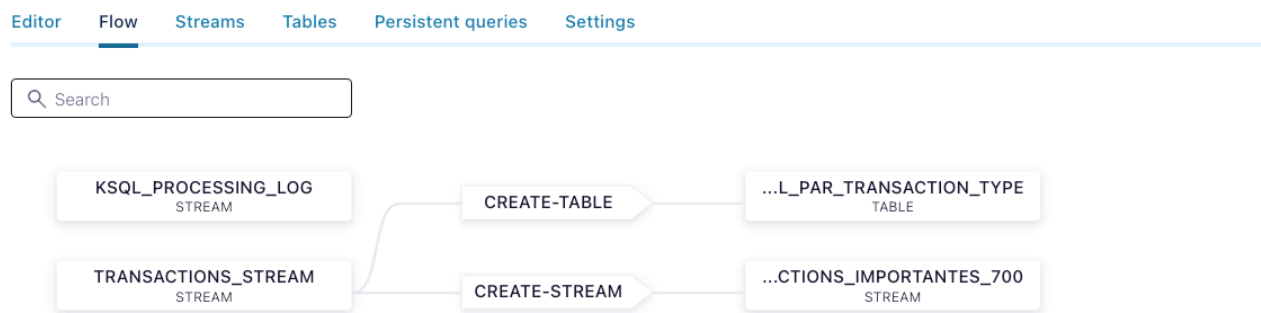
Une alternative est `CLEANUP_POLICY='compact'` Utile pour les cas d'utilisation où seule la dernière mise à jour d'une donnée est importante (Change Data Capture)

5- Création d'une table pour stocker le résultat d'un calcul

Total par type de transaction

```
CREATE TABLE total_par_transaction_type AS  
SELECT transaction_type, SUM(amount) AS total_amount  
FROM transactions_stream  
GROUP BY transaction_type;
```

Image du nouveau pipeline



6- Exécution du producer python

Exécutez le script python **kafka_producer_transaction.py** qui vous est fourni.

Il va injecter quelques messages dans le topic d'entrée de votre pipeline **transaction_log**

7- Exploration des résultats

Questions:

- Parmi les transactions importantes, quel est le type le plus représenté? Reportez votre requête SQL
- Quel est le schéma de la table TOTAL_PAR_TRANSACTION_TYPE (noms des colonnes et types) ? Reportez une capture d'écran ou alors la requête SQL pour retrouver l'information ainsi que le résultat.
- Parmi les transactions importantes, reportez le montant moyen ainsi que le montant médian. Reportez la requête SQL ou alors toute méthode utilisée.
- Pour le stream TRANSACTIONS_IMPORTANTES_700, de combien est la période de retention des données dans Kafka? Pourquoi cette période de retention est-elle nécessaire?